

Laboratoire 1

Les premiers pas

Objectifs d'apprentissage

- Créer votre première application Android avec Android Studio
- Comprendre l'architecture et la structure d'un projet Android
- Maîtriser les concepts fondamentaux des layouts et des composants d'interface
- Utiliser les identifiants pour interagir avec les éléments de l'interface
- Implémenter des interactions de base dans une application Android
- Distinguer les différentes méthodes de création de composants (XML, programmation, interface graphique)

1. Création d'une première application

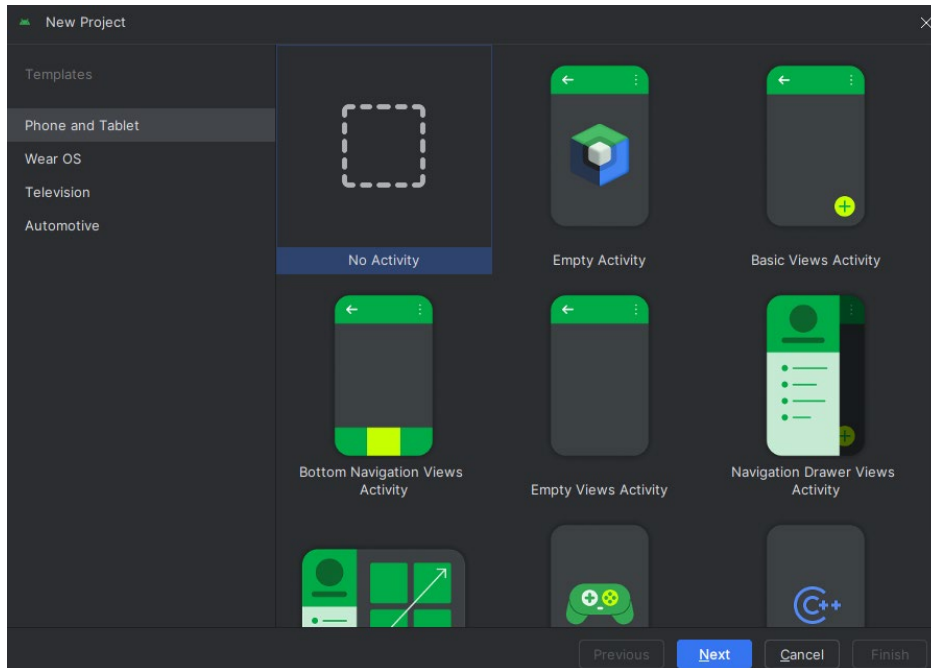
Étape 1: Initialisation du projet

a. Démarrer Android Studio

À l'ouverture d'Android Studio, cliquez sur le bouton **New Project** pour créer votre premier projet.

b. Choisir le modèle d'activité

Sélectionnez le modèle **No Activity** et cliquez sur **Next**. Cette approche vous permettra de mieux comprendre la structure d'un projet en créant l'activité manuellement.



c. Configuration du projet

Remplir les champs **Name** et **Package name** en suivant les indications de l'écran ci-dessous:

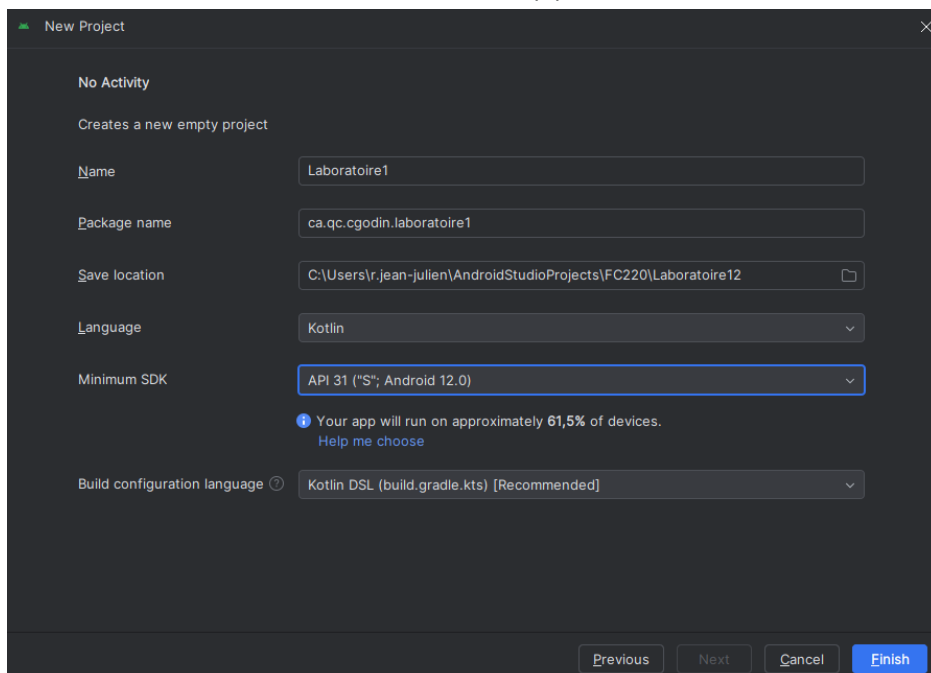
Name : Laboratoire1 (avec L majuscule)

Package name : ca.qc.cgodin.laboratoire1 (en minuscules, sans espaces)

Save location : Choisissez votre répertoire de travail

Langage : Kotlin

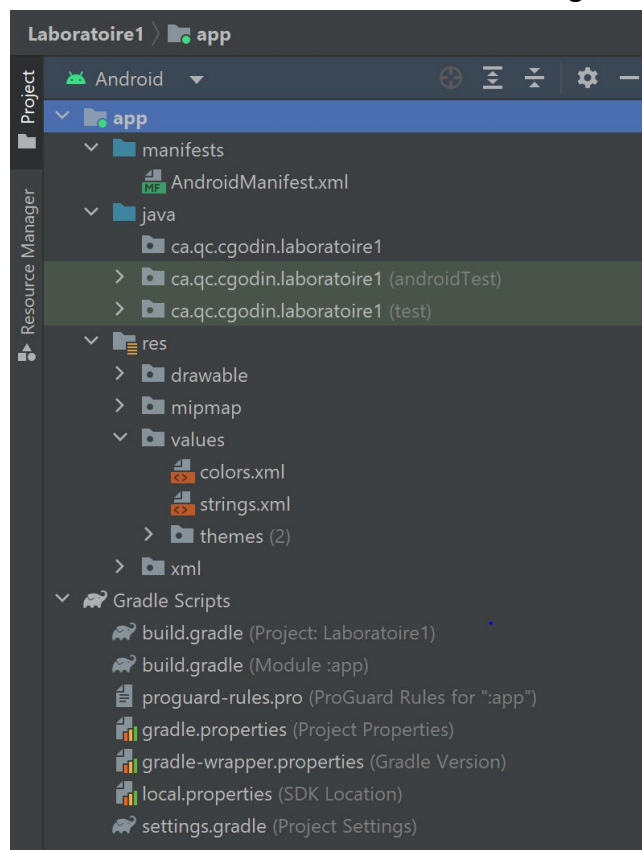
Minimum SDK : API 31 : Android 12.0 (S)



- **Important** : le nom de l'application doit commencer par une majuscule, car c'est ce nom qui apparaîtra pour l'utilisateur.
- Le "**Package name**" (par exemple, "**ca.qc.cgodin.laboratoire1**") est généré automatiquement. Noter que **laboratoire1** dans le nom du package doit commencer par une minuscule et ne contient aucun espace (conformément aux règles de nommage des packages en Kotlin). Le nom de package doit être unique et respecter les conventions de nommage Kotlin. Modifiez le nom en "**ca.qc.cgodin.laboratoire1**" comme indiqué ci-dessus si ce n'est pas le cas.
- **Project Location** : Choisissez votre emplacement de travail où vous souhaitez héberger votre projet (par défaut **C:\Users\<Votre compte>\AndroidStudioProjects**)

Cliquez sur **Finish**.

Android Studio va créer un répertoire avec un ensemble de fichiers pour le projet en utilisant **Gradle**. Ce processus peut prendre quelques minutes car **Gradle** configure l'environnement du projet. Un nouveau projet sera créé avec la structure arborescente suivante affichée à gauche de votre écran:

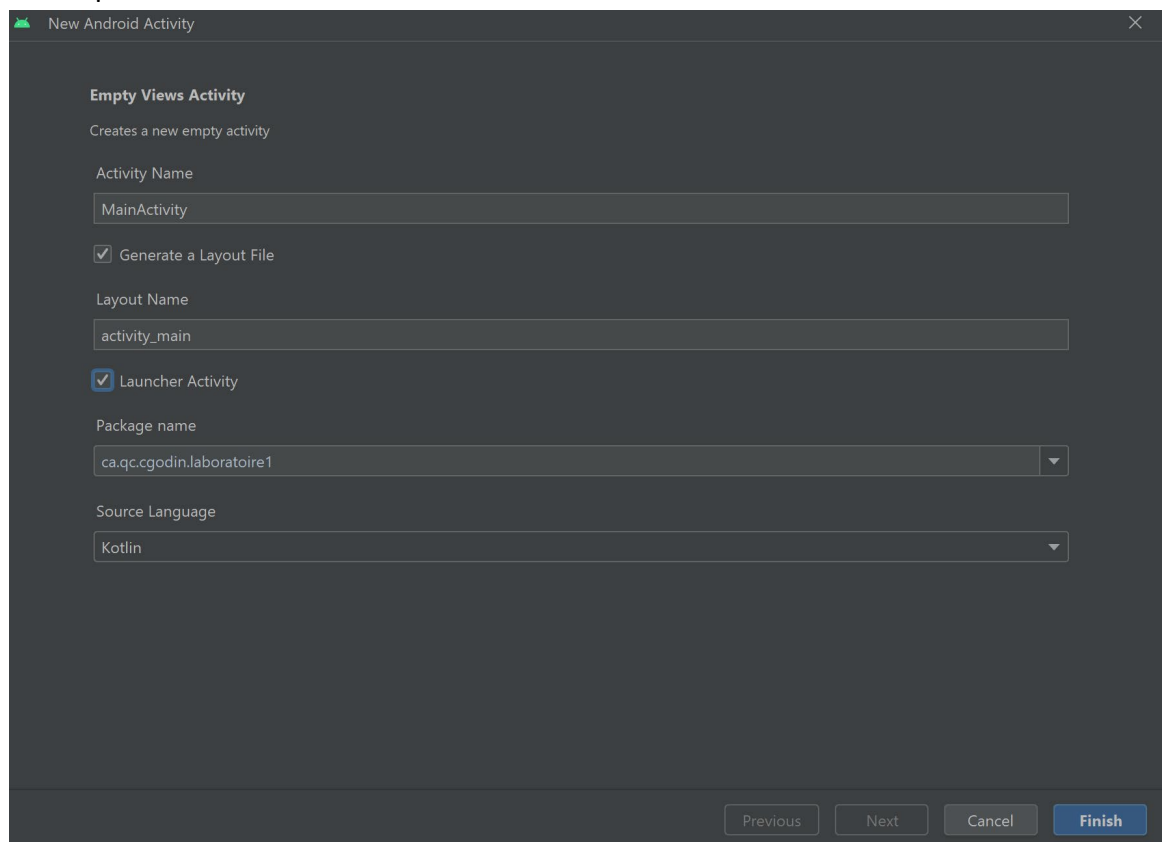


Étape 2 : Création de l'activité principale (votre première activité).

1. Créer MainActivity

Avant de créer votre première activité, vérifiez le contenu de votre fichier "**AndroidManifest.xml**".

- a. Dans le volet "**Projet**" à gauche, ouvrez le dossier **app > kotlin+java** et faites un clic droit sur le package "**ca.qc.cgodin.laboratoire1**". Sélectionnez "**New**" > "**Activity**" > "**Empty Views Activity**".
- b. Nommez l'activité "**MainActivity**" si ce n'est déjà fait. Cochez les options "**Generate a Layout File**" et "**Launcher Activity**". Cliquez sur "**Finish**". Cela créera automatiquement le fichier "**MainActivity.kt**" dans le package principal **ca.qc.cgodin.laboratoire1** et un fichier "**activity_main.xml**" dans un nouveau dossier **layout** qui sera créé dans le répertoire **res**.



- c. Vérifiez à nouveau le contenu de votre fichier **AndroidManifest.xml**. Notez bien que des modifications ont été apportées par rapport à la création de l'activité. Gardez à l'esprit que ce sera également le cas pour

tout ajout ultérieur de composant de base dans vos applications Android.

2. Exploration de la structure de votre projet

Comprendre l'organisation des fichiers

- a. Si l'onglet **Projet** n'est pas déjà sélectionné, sélectionner-le.
- b. L'onglet **Projet** se trouve dans le panneau vertical à gauche de votre fenêtre dans Android Studio.
- c. Cliquer sur l'onglet **activity_main.xml** pour afficher le fichier

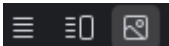
— Explorer le layout —

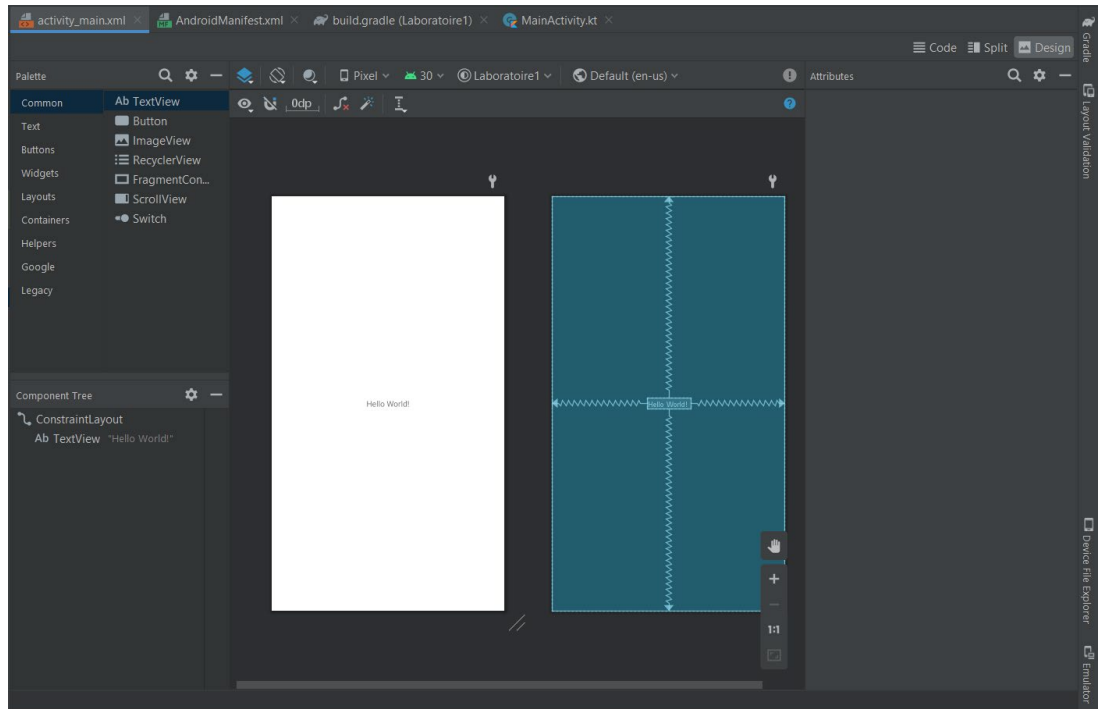
Ouvrez l'onglet activity_main.xml : c'est la mise en page (layout) de l'activité principale.

Trois modes d'affichage sont disponibles :

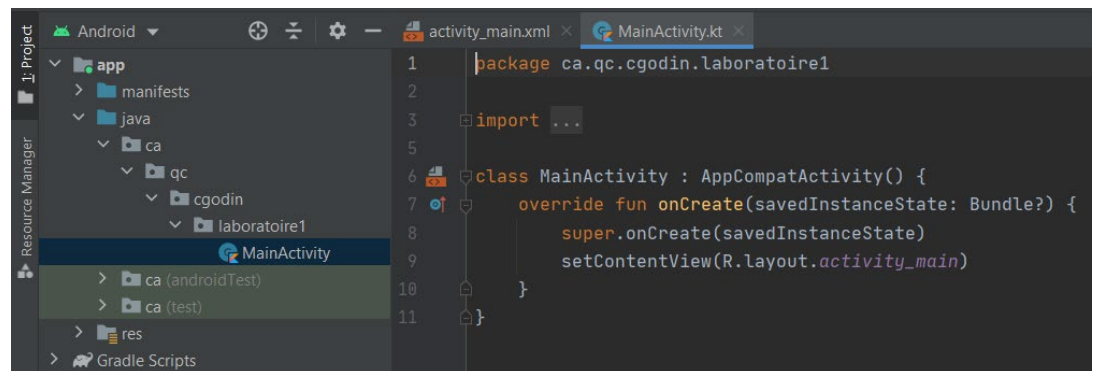
- Code : édition directe du XML (précis mais verbosité).
- Split : code + aperçu visuel (bon compromis).
- Design : éditeur visuel (glisser-déposer, rapide pour débiter).

Astuce : basculez entre les modes pour comprendre l'impact de vos modifications.

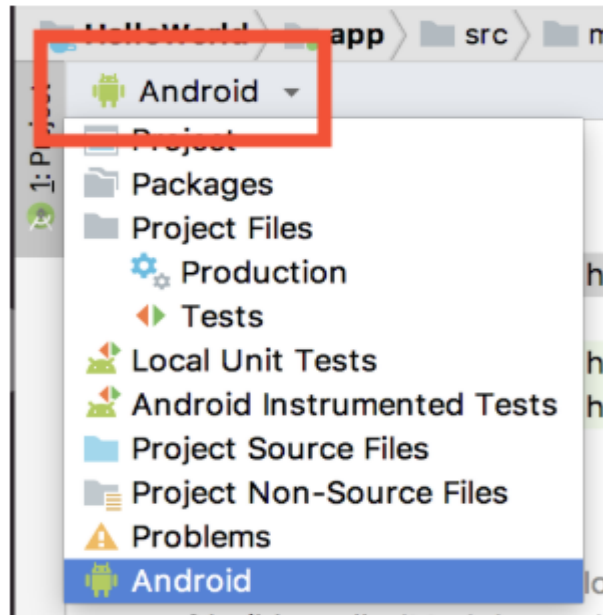
- d. Ensuite, cliquer sur les onglets "**Code - Split - Design**"  situés tout à droite pour visualiser leurs contenus. Observer la différence.



d. Cliquer sur l'onglet **MainActivity.kt** pour l'ouvrir dans l'éditeur.

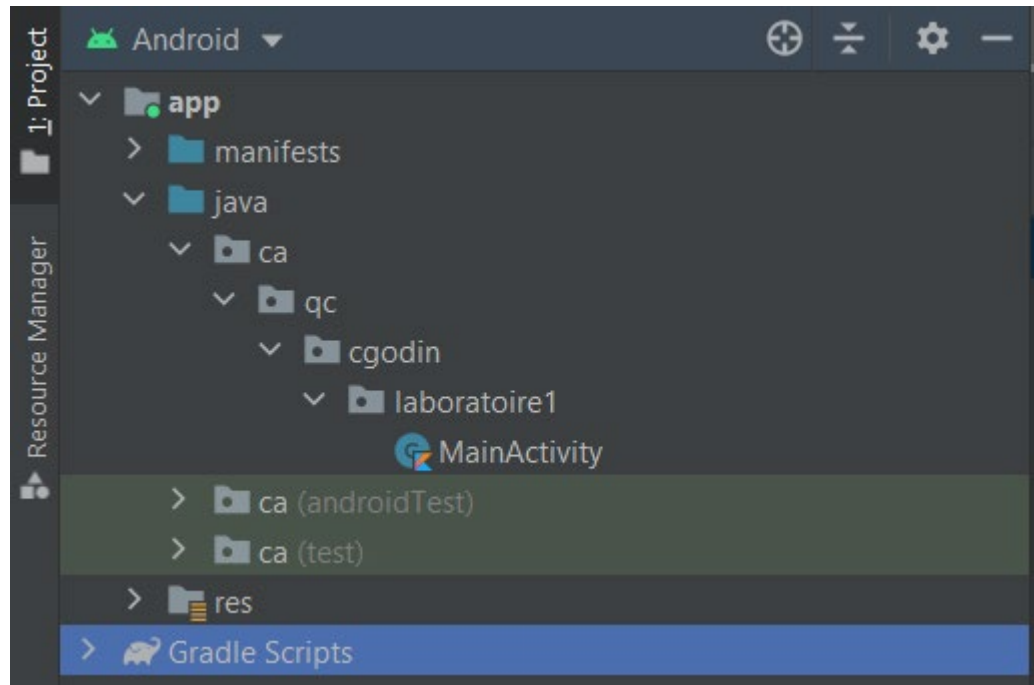


e. Pour afficher le projet comme une hiérarchie de projet Android standard, assurez-vous que l'option **Android** est sélectionnée dans le menu déroulant en haut du volet Projet. C'est généralement la valeur par défaut recommandée. Cette vue offre une meilleure compréhension du projet. Vous pouvez afficher les fichiers de projet de différentes façons, y compris l'affichage des fichiers comme ils apparaissent dans la hiérarchie du système de fichiers. Cependant, le projet est plus facile à travailler avec l'utilisation de la vue Android.



Exploration des dossiers de l'application

- a. Dans le volet **Projet** > **Android**, développez le dossier **app** de votre application. À l'intérieur de ce dossier, vous trouverez trois ou quatre sous-dossiers : **manifests**, **kotlin+java**, **res** et éventuellement **generatedJava**, qui contient des fichiers générés par Android Studio lors de la construction de l'application. Tout le code de votre application réside dans le répertoire **kotlin+java**, tandis que les ressources de votre application se trouvent dans le répertoire **res**. Les fichiers de configuration sont stockés dans le répertoire **manifests**.
- b. Ouvrez le fichier **app > java > ca.qc.cgodin.laboratoire1 > MainActivity.kt**.



Exploration du dossier kotlin+java

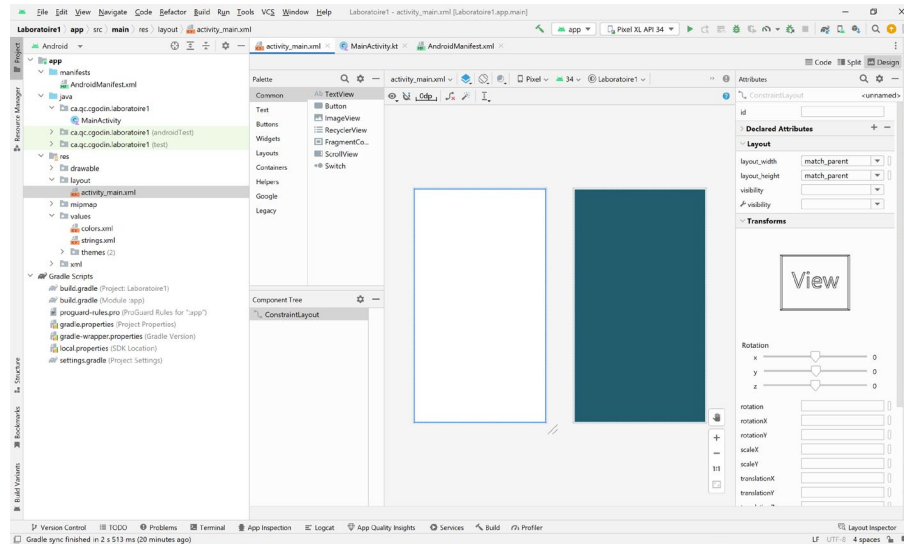
- Le dossier **kotlin+java** contient le code principal de votre application écrit en langage **Kotlin**.
- La présence du code Kotlin dans le dossier "kotlin+java" est une convention historique permettant une interaction transparente entre Kotlin et java dans un même projet. Les fichiers de classe de votre application sont organisés dans trois sous-dossiers, comme le montre la figure ci-dessus. Le package **ca.qc.cgodin.laboratoire1** contient actuellement le fichier **MainActivity.kt** qui agit comme porte d'entrée de votre application. Les deux autres packages sont généralement utilisés pour les tests.

Exploration du dossier res

À retenir : chaque activité est généralement liée à un XML dans res/layout (ex. activity_main.xml).

- Dans le volet **Project>Android**, développez le dossier **res** contenant les ressources de l'application.
- Le dossier **res > layout** abrite les descriptions d'interfaces graphiques définies dans des fichiers XML. Vous trouverez ici le fichier **activity_main.xml**, associé à l'activité principale.

- Ouvrez le fichier `activity_main.xml`. Vous devriez avoir un écran semblable à celui-ci ou du code XML. Une activité est en général associée à un fichier de **layout**, défini comme un fichier XML dans le répertoire **res/layout**.



Remarque sur les erreurs de rendu

Astuce : Essayez « Build → Rebuild Project ». Si le problème persiste, utilisez « File → Invalidate Caches / Restart ».

Si vous rencontrez des erreurs de rendu, telles que "Rendering problems: missing styles...", essayez les étapes suivantes

- Cliquer sur **rebuild** si cette option vous est proposée.
- Sinon, utiliser le menu **Fichier/Invalidate caches/restart**

Les ressources regroupent du contenu statique tel que des images, du texte, des styles, etc.

Les fichiers ressources sont organisés dans des sous-dossiers en fonction de leur type dont voici quelques sous-dossiers :

- mipmap-*dpi**: contient les images au format png, gif, jpeg et bmp.
- layout**: descriptions d'interfaces graphiques définies dans des fichiers au format xml. Noter la présence du fichier `activity_main.xml` pour notre première application.
- values**: valeurs de différents types simples (exemple : chaînes de caractères, entiers,...) contenus dans des fichiers `xml`. Chacune des valeurs est accessible via un identifiant unique.

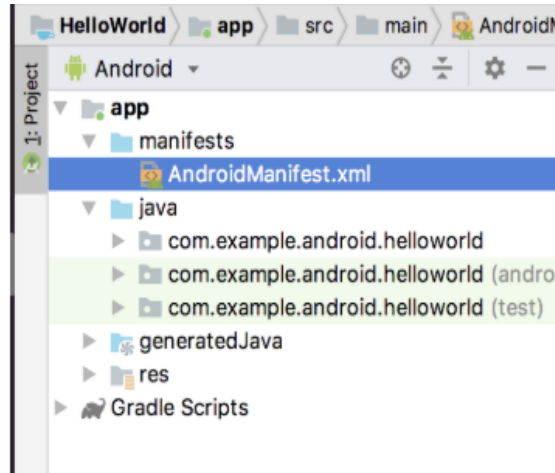
En outre, lorsque vous modifiez l'un de ces fichiers de ressources, la modification prend effet partout où le fichier est utilisé dans l'application.

Exploration du répertoire manifests et du fichier AndroidManifest.xml

1. Le dossier **manifests** contient des fichiers essentiels fournissant des informations importantes sur votre application.

Développez le dossier **manifests** et double-cliquez sur **AndroidManifest.xml** pour l'ouvrir.

Le fichier **AndroidManifest.xml** contient des détails nécessaires au système Android pour exécuter votre application, y compris les activités qui font partie. Chaque composant fondamental d'Android, comme une **Activité**, un **Service**, un **fournisseur de contenu** ou un **récepteur d'événements** doit être déclaré dans ce fichier.



2. Notez que **MainActivity** est référencé dans l'élément `<activity>`. Toute activité (et plus tard les Services, les Fournisseurs de Contenus (Content Provider), les récepteurs d'événements (Broadcast Receivers)) faisant partie de votre application doit être déclarée dans le fichier **manifest**. Vous aurez à utiliser les autres composants un peu plus tard dans le cours. Pour le moment, on se concentre sur le composant Activity. Voici un exemple de fichier **manifest**.

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:tools="http://schemas.android.com/tools">
4
5      <application
6          android:allowBackup="true"
7          android:dataExtractionRules="@xml/data_extraction_rules"
8          android:fullBackupContent="@xml/backup_rules"
9          android:icon="@mipmap/ic_launcher"
10         android:label="Laboratoire1"
11         android:roundIcon="@mipmap/ic_launcher_round"
12         android:supportsRtl="true"
13         android:theme="@style/Theme.Laboratoire1">
14         <activity
15             android:name=".MainActivity"
16             android:exported="true">
17             <intent-filter>
18                 <action android:name="android.intent.action.MAIN" />
19
20                 <category android:name="android.intent.category.LAUNCHER" />
21             </intent-filter>
22         </activity>
23     </application>
24
25 </manifest>

```

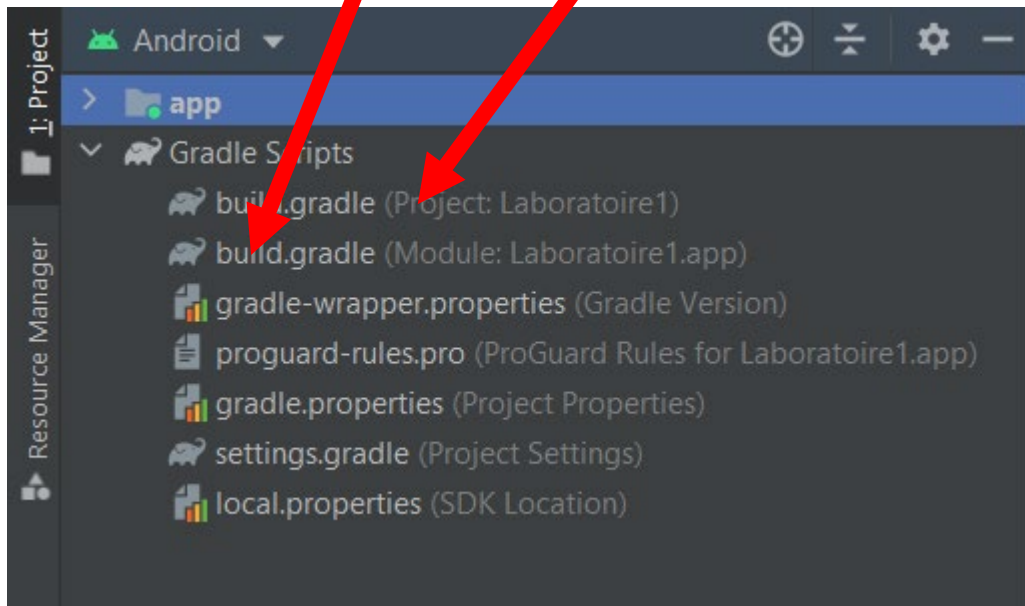
3. Notez l'élément `<intent-filter>` dans `<activity>`. Les éléments `<action>` et `<category>` indiquent à Android où commencer l'application quand l'utilisateur clique sur l'icône de démarrage.

Le manifeste ***AndroidManifest.xml*** est l'endroit où vous définirez également les permissions de votre application, telles que l'accès aux contacts ou à la caméra.

Explorer le répertoire app/build.gradle

Deux fichiers : build.gradle (Project) pour la configuration globale, et build.gradle (Module: app) pour les dépendances de l'application.

- a. Android Studio utilise **Gradle** pour compiler l'application. **Gradle** permet d'utiliser facilement des bibliothèques externes.
- b. Un fichier **build.gradle** est généré pour chaque module d'un projet Android.
- c. Un fichier **build.gradle** est aussi généré pour le projet au complet (le module d'application **app**). C'est ce fichier qui nous intéresse. Ce fichier contient les dépendances et les paramètres de configuration de l'application (compileSdkVersion, applicationId, minSdkVersion, targetSdkVersion). Vous aurez assez souvent à le modifier.



- d. Ouvrez le fichier **build.gradle (Project: Laboratoire1)**.
Vous allez trouver les configurations pour le projet au complet, pour tous les modules.
- e. Ouvrez le fichier **build.gradle (Module: app)**.
Chaque module dans un projet Android Studio doit contenir un fichier **build.gradle** qui permet de faire des configurations propres à ce module. Vous aurez souvent à manipuler ces fichiers pour ajouter des dépendances à vos modules.

3. Accès aux ressources

Votre projet contient des répertoires contenant des fichiers auto-générés. Il est mis à jour automatiquement quand on ajoute ou supprime des ressources au projet (exemple : ajout d'un composant graphique). **Il ne faut jamais modifier ces fichiers.**

Accès aux ressources d'un projet

Les différentes ressources de votre projet se trouvent dans le répertoire `res` (mis pour ressources). Vous pouvez accéder à ces fichiers soit à partir d'un fichier Java/Kotlin (**Ex : MainActivity.kt**), soit à partir d'un fichier XML (**Ex : activity_main.xml**).

Accès aux ressources à partir d'un fichier Java ou kotlin

`[package].R.type.nom`

- `type` est le nom du sous-dossier du dossier **res** contenant la ressource.
- `nom` est le nom du fichier sans extension.

Exemples :

`R.layout.activity_main` (Voir fichier MainActivity.kt)

Cette syntaxe fonctionne pour toutes les ressources à l'exception de celles du dossier **res/values**. Dans ce cas, le type de la ressource est le nom de la balise définissant la ressource et le nom de la ressource est le nom spécifié par l'attribut **name** de la balise de la ressource.

Exemples :

`R.string.app_name` (défini dans le fichier strings.xml)

`R.color.white` (défini dans le fichier colors.xml)

Accès aux ressources à partir d'un fichier xml

`@[package:]type/nom`

Exemples :

`@mipmap/ic_launcher`

`@layout/activity_main`

`@string/app_name`

`@color/white`

4. Exécution de l'application

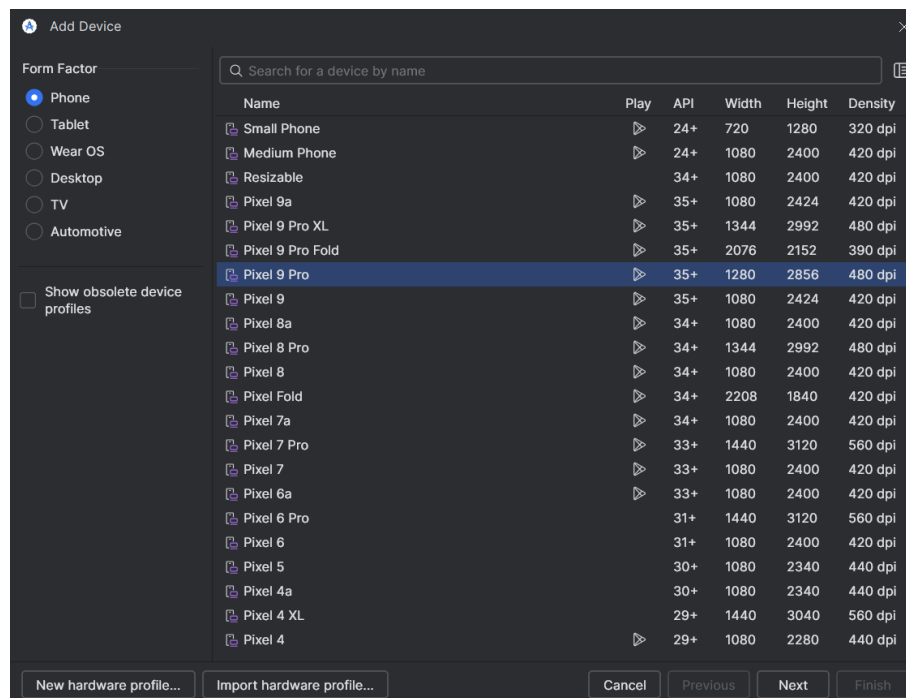
En résumé (AVD) : Device Manager → + Create Virtual Device → choisir un profil → Finish → ► Exécuter.

Appareil physique : activer les options développeur et le débogage USB, puis lancer l'exécution.

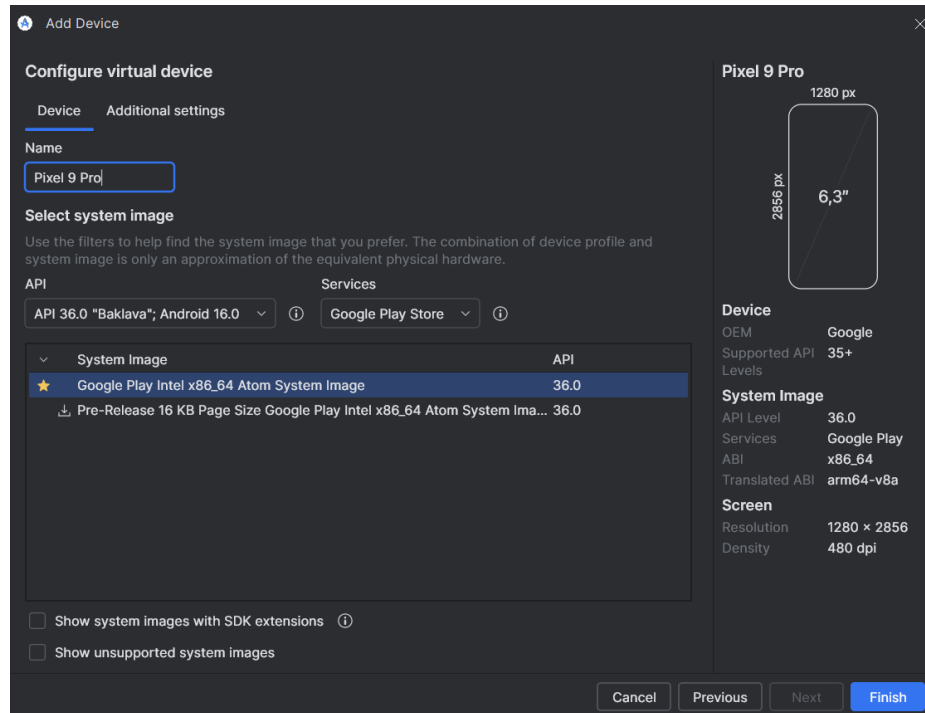
a. Dans un environnement simulé

Le SDK propose un émulateur **Android Virtual Device (AVD)**, permettant de tester les applications en local pour différents types de périphériques.

Dans Android Studio, sélectionnez Tools->Device Manager, ou cliquez sur le bouton Device Manager dans la barre d'outils. Cliquez sur le bouton + > "**Create virtual Device**" dans le nouveau panneau à droite pour créer un périphérique virtuel. Choisissez Pixel 9 Pro et cliquez sur Next.

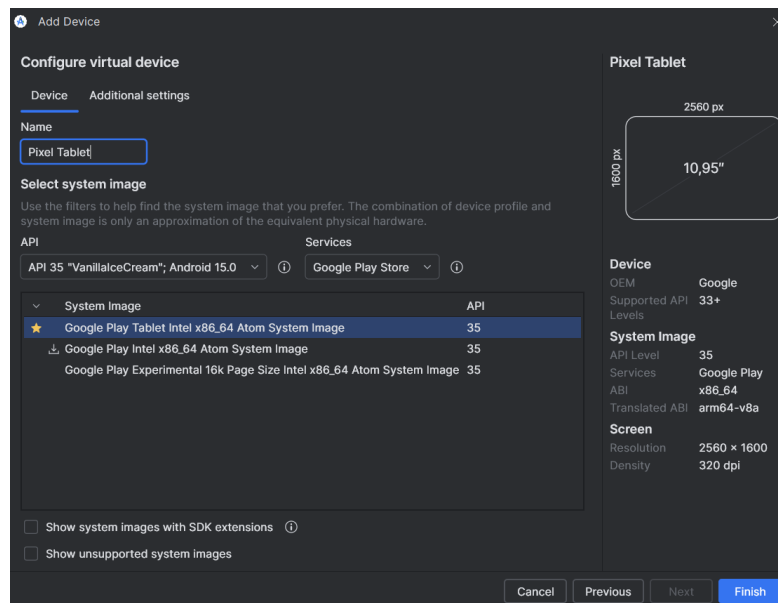


Sélectionnez les options suivantes dans le nouveau panneau:



Laissez les options sélectionnées par défaut et cliquer sur **Finish** pour créer votre téléphone virtuel.

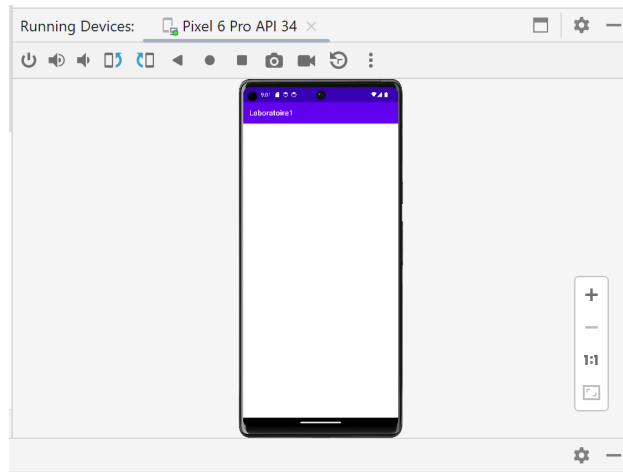
Suivez la même approche pour créer une tablette virtuelle (Name: **Pixel Tablet**)



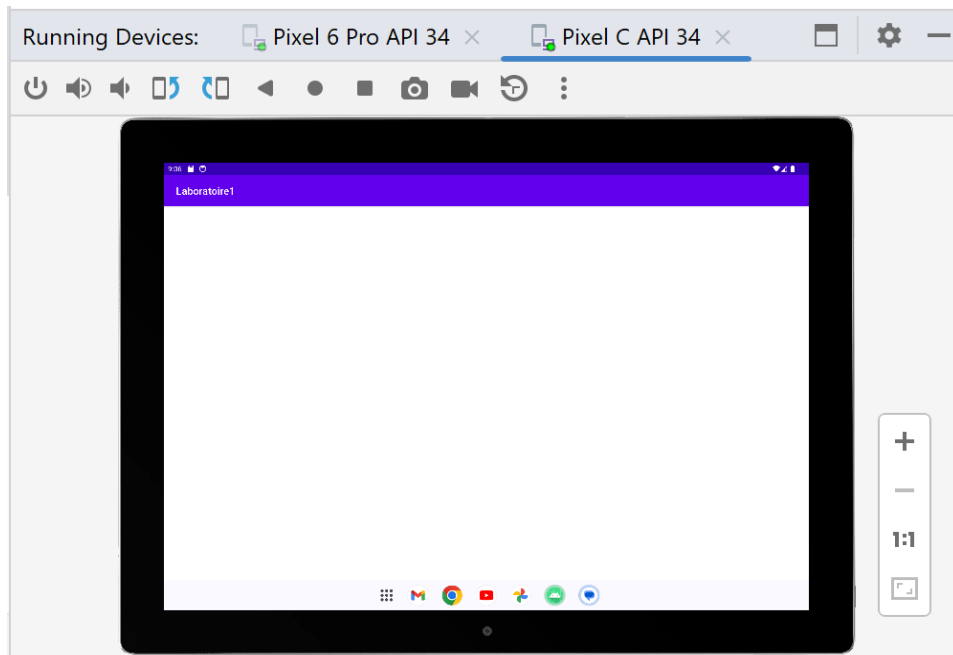
Cliquez sur **Run->Run App** ou cliquez sur le bouton **Run** dans la barre d'outils pour exécuter l'application. Sélectionnez le téléphone dans un premier temps et ensuite la tablette dans la barre d'outils pour exécuter votre application sur les deux périphériques.

Les résultats des exécutions devraient ressembler à ceci :

Téléphone:



Tablette:



b. Directement sur un téléphone ou une tablette

Dans ce cas, si vous avez un appareil intelligent (smartphone ou tablette Android), il faut connecter votre appareil intelligent à l'ordinateur via un câble USB. Il faut s'assurer que le pilote adéquat a été installé auparavant. Dans **Paramètres/Options de développement** de votre tablette ou de votre téléphone, vérifier que débogage **USB est coché**. Sinon dans **Paramètres->À**

propos du téléphone (de la tablette)->Informations sur le logiciel, appuyez 7 fois sur **Numéro de version (Build Number)**. Vous devez recevoir un message disant que le mode développeur a été activé.

Cliquez sur **Run->Run App** ou cliquez sur le bouton **Run** dans la barre d'outils pour exécuter l'application. Sélectionnez le téléphone ou la tablette pour exécuter l'application.

5. Modification de l'application

- Dans le dossier ressource (**res/layout**), double-cliquez sur **activity_main.xml**. Noter la présence des trois onglets (**Code, Split et Design**). Vérifier que c'est l'onglet **Design** qui est activé.
- Notez la présence de l'arborescence des composants (**Component tree**) qui se trouvent dans l'éditeur de **layout** en bas à gauche.
- Ajouter un **textView** avec le texte "**Bienvenue au cours 420-P5C!**" à votre interface.

Dans la palette des composants, trouver le composant **TextView** dans la section Common (ou chercher **TextView** dans la barre de recherche de la palette).

Faire un glisser-déposer du composant **TextView** dans le centre de l'écran. Vous verrez un rectangle représentant le TextView apparaître avec comme texte de départ **TextView**.

- **Centrer le TextView dans le ConstraintLayout**

Pour centrer le TextView horizontalement et verticalement, il faut lui ajouter des contraintes (constraints) pour le relier au conteneur parent (le **ConstraintLayout** ayant pour id **main**).

- a. Sélectionner le **TextView** :

Cliquer sur le **TextView** dans la vue **Design**. Il sera entouré de poignées (petits ronds sur les bords).

- b. **Ajouter des contraintes horizontales** :

Cliquez sur la poignée **gauche** du TextView et faites-la glisser vers le bord **gauche** du ConstraintLayout. Une ligne apparaîtra pour indiquer la contrainte.

Faites de même pour la poignée **droite**, en la reliant au bord **droit** du ConstraintLayout.

Résultat : Le TextView est maintenant contraint horizontalement aux deux côtés du layout.

- c. **Ajouter des contraintes verticales** :

Cliquez sur la poignée de **haut** du TextView et reliez-la au bord **supérieur** du ConstraintLayout.

Faites de même pour la poignée de **bas**, en la reliant au bord **inférieur** du ConstraintLayout.

- Utilisez le panneau **Attributes** pour mettre la valeur de l'attribut **id** du TextView à **tvBienvenue** (pour identifier la zone de texte). Appuyez sur **Entrée** et cliquez sur **Refactor**.
- Dans le panneau Attributes > Layout > Common Attributes, modifiez la valeur de l'attribut **text** à **Bienvenue au cours 420-P5C!**, la valeur de l'attribut **textSize** à 24sp, la valeur de l'attribut **textColor** du texte à #FF0000.

Note : Ce n'est pas une bonne pratique de placer directement une chaîne de caractères comme valeur pour les attributs des éléments. Pour corriger ce problème, nous allons placer les chaînes dans les fichiers strings.xml pour les chaînes de caractères, colors.xml pour les couleurs, dims.xml (que vous devez ajouter dans le répertoire values) pour les dimensions et y faire référence.

Définissez la couleur rouge dans le fichier **values/colors.xml** en lui ajoutant la ligne suivante avant la fermeture de la balise **</resources>**

```
<color name="rouge">#FF0000</color>
```

Définissez la chaîne welcome_msg dans le fichier **values/strings.xml** en lui ajoutant la ligne suivante avant la fermeture de la balise **</resources>**

```
<string name="welcome_msg">Bienvenue au cours 420-P5C! </string>
```

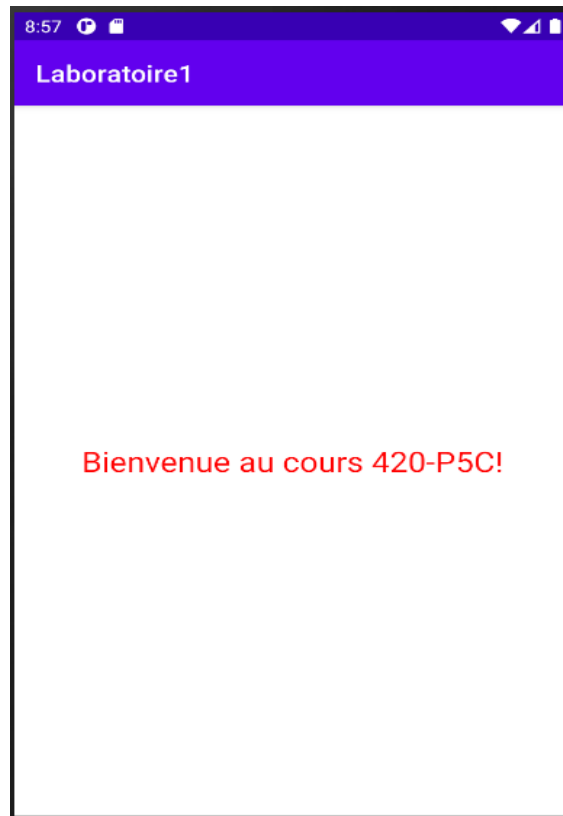
- Remplacer dans le fichier de layout activity_main.xml **android:text="Bienvenue au cours 420-P5C!"** par **android:text="@string/welcome_msg"**.

Remarquez bien que vous avez un message d'erreur. Pour que cette valeur soit effective pour l'attribut **android:text**, il faut ajouter la variable **welcome_msg** dans le fichier strings.xml.

Dans le dossier **res/values**, double-cliquez sur **strings.xml** et ajoutez la ligne suivante avant la fermeture de la balise **</resources>**:

```
<string name="welcome_msg">Bienvenue au cours 420-P5C!</string>
```

- Relancez l'application et notez les changements dans l'interface graphique.



1. Ajout d'une zone de texte (TextView) avec des images en utilisant XML

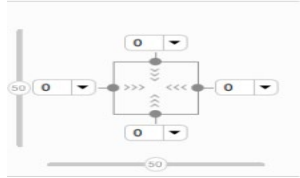
Ouvrez le fichier `activity_main.xml` et cliquez sur l'onglet Code pour voir le code XML. Notez que le **Layout** créé par défaut par Android Studio est un **ConstraintLayout**. Les composants y sont disposés les uns par rapport aux autres.

- Utilisez la barre d'outils pour mettre la marge par défaut à 16dp. (La valeur par défaut étant de 0dp.)



Lorsque vous mettez la marge par défaut à 16dp, les futures contraintes que vous ajouterez plus tard seront créées avec cette marge. Vous n'aurez donc pas besoin d'ajouter la marge à chaque fois que vous ajoutez une contrainte.

- Cliquez sur le **textView** en mode Design et observez l'inspecteur de vue suivant dans le volet **Attributes**.



L'inspecteur de vue comprend des commandes pour les attributs de mise en page tels que les contraintes, les types de contraintes, le biais de contrainte et les marges de vue. Cet inspecteur n'est disponible que pour les vues compris dans un **ConstraintLayout**.

- Biais de contrainte (Constraint bias)
Le biais de contrainte (Constraint bias) positionne l'élément de vue le long des axes horizontal et vertical. Par défaut, la vue est centrée entre les deux contraintes avec un biais de 50 %.

Pour ajuster le biais, vous pouvez faire glisser les curseurs de

biais  dans l'inspecteur de vue. Faire glisser un curseur de biais modifie la position de la vue le long de l'axe.

- Ajoutez des marges au textView
Notez que dans l'inspecteur de vue, les marges gauche, droite, supérieure et inférieure de la vue texte sont 0. La marge par défaut n'a pas été ajoutée automatiquement, car cette vue a été créée avant que vous ne modifiiez la marge par défaut.

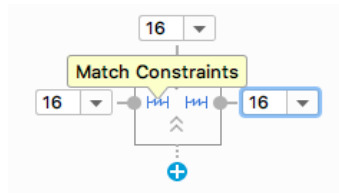
Pour les marges gauche, droite et supérieure, sélectionnez **16dp** dans le menu déroulant de l'inspecteur de vue. Par exemple, dans la capture d'écran suivante, vous ajoutez `layout_marginTop`, `layout_marginStart` `layout_marginEnd`.

- Ajouter des contraintes et des marges au textView

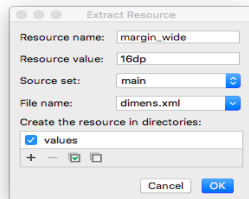
Dans l'inspecteur de vue, les flèches **>>>** à l'intérieur du carré représentent le type de contrainte :

- **>>> Wrap-Content** : la vue s'étend uniquement autant que nécessaire pour contenir son contenu.
- **Fixed** : vous pouvez spécifier une dimension comme marge de vue dans la zone de texte à côté des flèches de contrainte fixe.

-  **Match Constraints** : la vue s'agrandit autant que possible pour répondre aux contraintes de chaque côté, après avoir pris en compte les propres marges de la vue. Cette contrainte est très flexible, car elle permet à la mise en page de s'adapter aux différentes tailles et orientations d'écran. En laissant la vue correspondre aux contraintes, vous avez besoin de moins de mises en page pour l'application que vous créez.
1. Dans l'inspecteur de vue, remplacez les contraintes de gauche et de droite par **Match Constraints**  . (Cliquez sur la flèche pour basculer entre les types de contraintes.)



2. Dans l'inspecteur de la vue, cliquez sur le point **Delete Bottom Constraint**  dans le carré pour supprimer la contrainte.
3. Passez à l'onglet **Code**. Extrayez la dimension correspondant au `layout_marginStart` dans la ressource auquel vous donnez le nom `margin_wide` (Faites pointer la souris sur la valeur 16dp, une ampoule jaune apparaît. Cliquez sur l'ampoule et sélectionnez **Extract dimension resource**.)



Faites la même approche pour la marge supérieure (top) et la marge de droite (end).

Dans le code XML, vous devez avoir ceci:

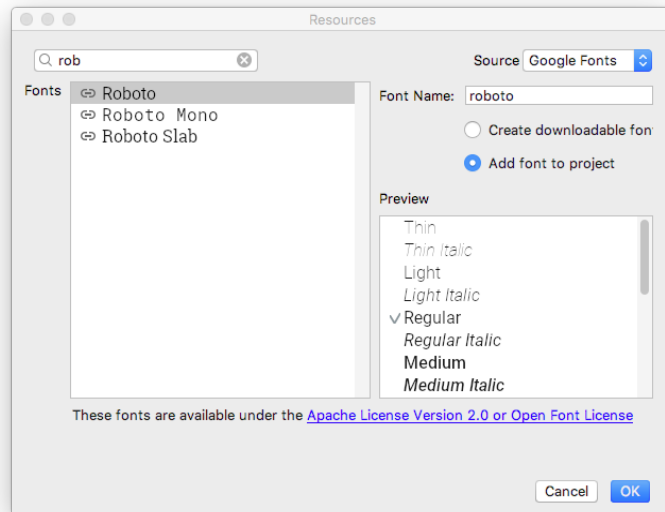
```
android:layout_marginStart="@dimen/margin_wide"
android:layout_marginTop="@dimen/margin_wide"
android:layout_marginEnd="@dimen/margin_wide"
```

- Ajouter du style au TextView

Ajouter un *font* au TextView

Dans le panneau **Attributes**, faites une recherche sur *fontFamily* et sélectionnez la liste déroulante à  côté. Faites défiler jusqu'à **More Fonts...** et sélectionnez-le. Dans la boîte de dialogue **Ressources** qui s'ouvre, recherchez **roboto**. Cliquez sur **Roboto** et sélectionnez **Regular** dans la liste **Preview**, sélectionnez le bouton radio **Add font to project**.

Cliquez sur **OK**.



Un dossier **res/font** contenant un fichier **roboto.ttf** sera ajouté au projet et l'attribut `@font/roboto` sera appliquée au `textView`.

Ajout de style au `textView`

Ouvrez le fichier `res/values/dimens.xml` et ajoutez la ligne suivante définissant une taille de police de 24sp.

```
<dimen name="box_text_size">24sp</dimen>
```

Ouvrez le fichier **`res/values/themes.xml`** et ajoutez le style suivant que vous aurez à utiliser dans le **`textView`**.

```
<style name="whiteBox">
  <item name="android:background">@android:color/holo_blue_dark</item>
  <item name="android:textAlignment">center</item>
  <item name="android:textSize">@dimen/box_text_size</item>
  <item name="android:textStyle">bold</item>
  <item name="android:textColor">@android:color/white</item>
  <item name="android:fontFamily">@font/roboto</item>
</style>
```

Dans ce style, la couleur d'arrière-plan(**`android:background`**) et la couleur du texte (**`android:textColor`**) sont définies sur les ressources de couleur par défaut fournies par Android. La police(**`android:fontFamily`**) est définie sur ***Roboto***. Le texte est centré et en gras, et la taille du texte est `box_text_size` (24sp).

- **Ajoutez le texte du `textView` dans `strings.xml`**
- Recherchez l'attribut **`text`** dans le panneau ***Attributes***. (Choisissez celui-là sans icône de clé)

Cliquez sur la petite barre latérale complètement à droite de l'attribut **text** pour ouvrir la boîte de dialogue **Resources**.

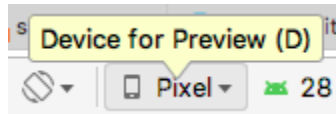
Cliquez sur le bouton "+" > "**String value**". Ajoutez la chaîne de caractère "**Cégep Gérald Godin**" dans le champ Resource value avec comme nom de ressource **txt_cgodin** dans le champ Resource name. Cliquez sur **OK**.

Donnez l'id **tvCGodin** au textView.

Dans la fenêtre **Attributes**, changez la valeur de l'attribut **style** à **@style/whiteBox**.

Pour nettoyer le code, supprimez dans le fichier activity_main.xml la valeur `android:fontFamily="@font/roboto"` appliqué au TextView.

Mettez-vous en mode design. Dans la barre d'outils, cliquez sur le bouton "**Device for Preview**" avec la valeur **Pixel** par défaut, vous devez voir une liste de périphériques parmi lesquels le périphérique **Pixel** qui est utilisé par défaut.



Sélectionnez différents périphériques dans la liste et observez l'adaptation du TextView aux différentes configurations d'écran.

Exécutez votre application. Vous devez voir une fenêtre similaire à celle-ci.

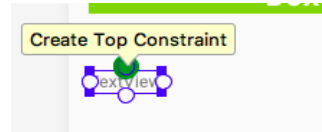


- Ajout d'un deuxième textView à notre interface

Ouvrez le fichier **activity_main.xml** en mode Design

Faites glisser un **TextView** du panneau Palette sous le premier **textView**

Cliquez sur le nouveau textView, puis maintenez le pointeur le point supérieur situé au centre comme indiqué ci-dessous. On appelle ce point une poignée de contrainte. Vous allez utiliser ce point pour créer une contrainte reliant la partie supérieure du nouveau textView au bas de l'ancien TextView.



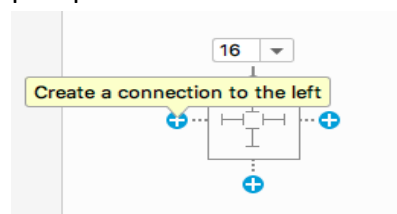
Cliquez sur la poignée de contrainte supérieure du nouveau **textView** et faites-la glisser vers le haut. Vous devez voir apparaître une ligne de contrainte. Connectez la ligne de contrainte à la poignée de contrainte inférieure du premier **textView**. Remarquez que la poignée de contrainte supérieure du nouveau **textView** se remplit automatiquement une fois la contrainte établie. Modifiez la marge supérieure à 16dp si ce n'est pas déjà fait.

Créez maintenant une contrainte de gauche:

Cliquez sur la poignée de contrainte sur le côté gauche du nouveau **textView**.

Faites glisser la ligne de contrainte complètement vers le côté gauche de votre interface.

Vous pourriez aussi créer les contraintes en cliquant sur les icônes  dans l'inspecteur de vue comme indiqué ci-dessous. Lorsque vous créez une contrainte de cette façon, la contrainte est attachée au parent ou à une vue plus proche.



Ajoutez la ligne suivante au fichier strings.xml où votre_nom correspond à votre nom.

```
<string name="votre_nom">Gérald Godin</string>
```

Modifiez les attributs du nouveau TextView comme suit:

Attribut	Valeur
id	tvName
layout_height	130dp
layout_width	130dp

style	@style/whiteBox
text	@string/votre_nom

Important : lors du développement d'applications du monde réel, utilisez des contraintes flexibles pour la hauteur et la largeur de vos éléments d'interface utilisateur, dans la mesure du possible. Par exemple, utilisez **match_constraint** ou **wrap_content**. Plus vous avez d'éléments d'interface utilisateur de taille fixe dans votre application, moins votre mise en page est adaptative pour différentes configurations d'écran.

Exécutez votre application. Vous devez avoir une fenêtre similaire à la capture suivante:



- Création d'une chaîne de 3 textViews

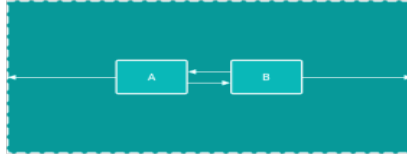
Vous allez maintenant ajouter une chaîne de 3 textViews à côté du deuxième textView disposés de façon verticale l'un après l'autre. Les textViews feront partie d'une chaîne.

Chaîne: Définition

Définition : une chaîne regroupe des vues reliées par des contraintes réciproques ; la première vue est la tête de chaîne.

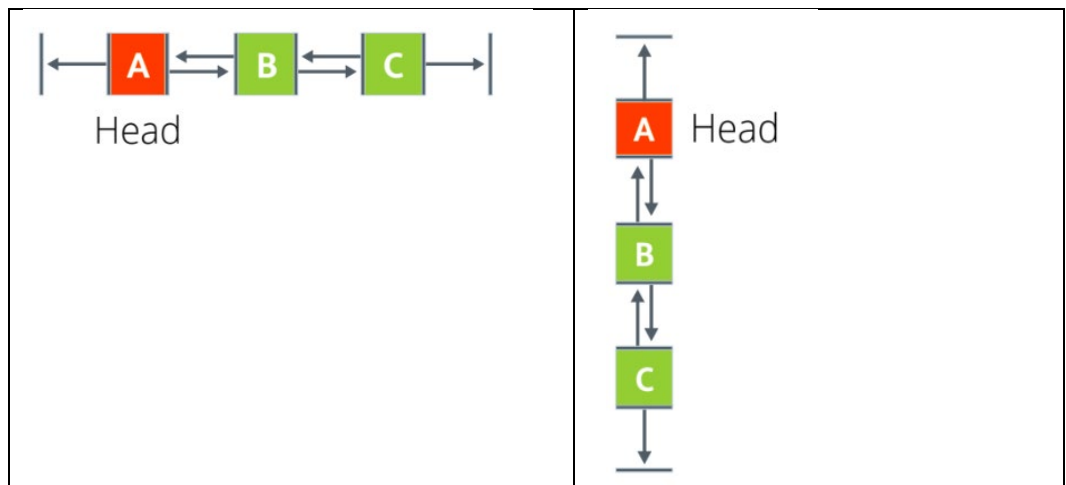
Styles : spread (espacé), spread_inside (extrémités ancrées), packed (regroupé), pondéré (répartition selon un poids).

Une chaîne est un groupe de vues liées les unes aux autres par des contraintes bidirectionnelles. Les vues au sein d'une chaîne peuvent être réparties verticalement ou horizontalement. Par exemple, le diagramme suivant montre deux vues contraintes l'une à l'autre, ce qui crée une chaîne horizontale.



Tête de chaîne

La première vue d'une chaîne s'appelle la *tête* de la chaîne. Les attributs définis sur la tête de la chaîne contrôlent, positionnent et distribuent toutes les vues de la chaîne. Pour les chaînes horizontales, la tête est la vue la plus à gauche. Pour les chaînes verticales, la tête est la vue la plus haute. Dans chacun des deux diagrammes ci-dessous, "A" est la tête de la chaîne.



Style de chaîne

Les styles de chaîne définissent la manière dont les vues chaînées sont réparties et alignées. Vous stylisez une chaîne en attribuant un attribut de style à cette chaîne, en ajoutant un poids ou en définissant un biais sur les vues.

Il existe trois styles de chaînes :

- **Spread** : Il s'agit du style par défaut. Les vues sont réparties uniformément dans l'espace disponible, une fois les marges prises en compte.



- **Spread inside** : la première et la dernière vue sont attachées au parent à chaque extrémité de la chaîne. Les autres vues sont réparties uniformément dans l'espace disponible.



- **Packed** : les vues sont regroupées ensemble, après application des marges. Vous pouvez ensuite ajuster la position de toute la chaîne en modifiant le de la tête de la chaîne.



- **Weighed (Pondéré)** : les vues sont redimensionnées pour remplir tout l'espace, en fonction des valeurs définies dans les attributs [layout_constraintHorizontal_weight](#) ou [layout_constraintVertical_weight](#). Par exemple, imaginez une chaîne contenant trois vues, A, B et C. La vue A utilise un poids de 1. Les vues B et C utilisent chacune un poids de 2. L'espace occupé par les vues B et C est le double de celui de la vue A , comme indiqué ci-dessous.



Pour ajouter du style à une chaîne, attribuez une valeur à l'attribut [layout_constraintHorizontal_chainStyle](#)/[layout_constraintVertical_chainStyle](#) de la tête de la chaîne. Vous pouvez ajouter des styles de chaîne dans l'éditeur de Layout. Vous pouvez en faire de même dans le fichier XML.

Par exemple:

```
// Horizontal spread chain
app:layout_constraintHorizontal_chainStyle="spread"

// Vertical spread inside chain
app:layout_constraintVertical_chainStyle="spread_inside"

// Horizontal packed chain
app:layout_constraintHorizontal_chainStyle="packed"
```

Ajout des 3 vues et création de la chaine verticale

Ouvrez le fichier `activity_main.xml` en mode Design. Faites glisser 3 TextView à partir du panneau Palette à la droite du deuxième textView comme suit.



Ajoutez les chaînes suivantes au fichier `strings.xml`.

```
<string name="txt_cours1">Android</string>
<string name="txt_cours2">ASP.Net</string>
<string name="txt_cours3">PHP</string>
```

Modifiez les attributs des trois textView suivant ce tableau:

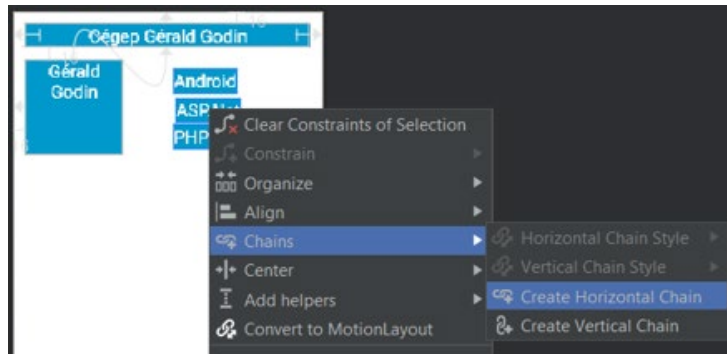
Atributs	textView3(Android)	textView4(ASP.net)	textView5(PHP)
id	tvCours1	tvCours2	tvCour3
text	@string/txt_cours1	@string/txt_cours2	@string/txt_cours3
style	@style/whiteBox	@style/whiteBox	@style/whiteBox



Vous devez voir présentement des erreurs de contraintes que vous aurez à corriger un peu plus tard.

Créez maintenant une chaîne avec les 3 **textView**s et contraignez la chaîne à la hauteur du deuxième **textView**.

Sélectionnez ensemble les trois nouveaux **textView**s. Faites ensuite un clic droit, sélectionnez **Chains > Create Vertical Chain**. Une chaîne verticale sera créée automatiquement s'étendant du textView1 au pied de page.



Ajoutez une contrainte s'étendant du haut du **textView3** au sommet du **textView2**. La contrainte supérieure existante sera automatiquement remplacée par la nouvelle contrainte. Vous n'avez pas à supprimer cette contrainte de façon explicite.

Ajoutez une contrainte du bas du **textView5** au bas du **textView2**.

Observer que les 3 textViews sont maintenant limités en haut et en bas par rapport au textView2.

Ajout de contraintes gauche et droite

Contraignez le côté gauche du textView3 au coté droit du textView2. Faites-en de même pour textView4 et textView5.

Contraignez le côté droit des trois derniers **textViews** au côté droit de l'interface.

Modifiez la valeur de l'attribut **layout_width** des trois derniers **textViews** à **0dp**, ce qui équivaut à **Math Constraints**.

Ajout de marge

Utilisez le panneau Attributes pour modifier les marges des 3 derniers textViews comme suit. Utilisez la valeur `@dimen/margin_wide` pour ces marges.

textView3: début et fin (start et end). Supprimer les autres marges.

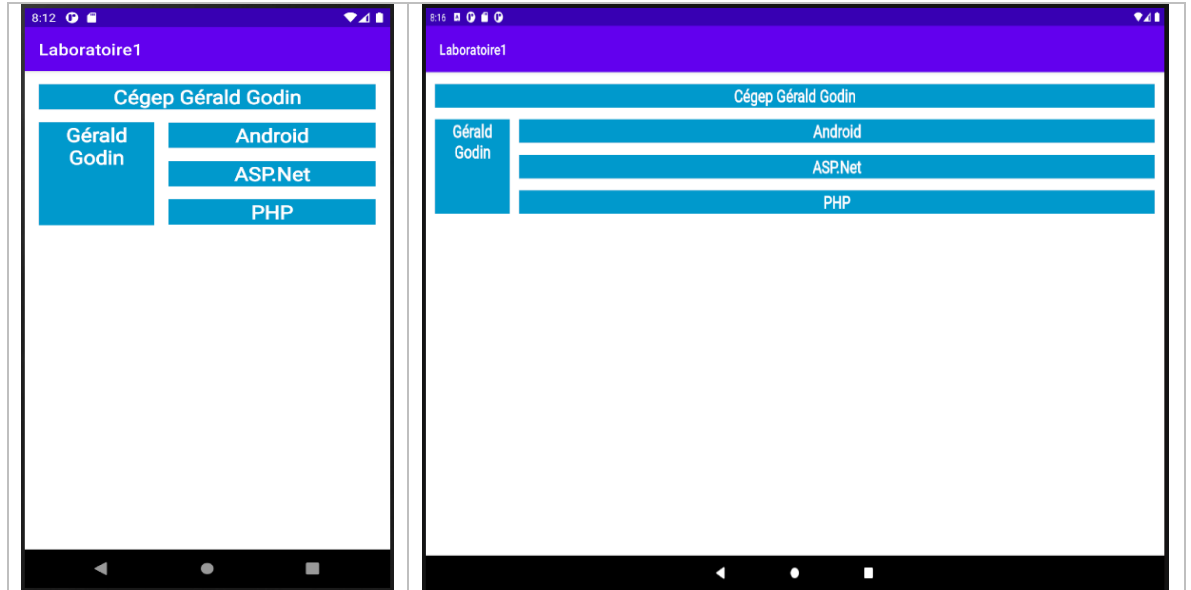
textView4: début et fin, haut et bas (start et end, top et bottom). Supprimer les autres marges.

textView5: début et fin (start et end). Supprimer les autres marges.

Pour voir maintenant comment les textViews s'adaptent aux changements de configuration de votre périphérique, modifiez l'orientation de l'aperçu.

Pour ce faire, cliquez sur l'icône **Orientation for Preview (O)**  dans la barre d'outils et sélectionnez **Landscape (Paysage)**.

Exécutez l'application sur un émulateur de tablette et un émulateur de téléphone. Vous devriez voir les 5 **textViews** affichés comme sur les captures ci-dessous.



Ajouter des gestionnaires d'événements aux textViews

Dans cette partie, vous allez modifier l'application pour la rendre un peu plus dynamique et colorée. Vous allez dans un premier temps mettre l'arrière-plan des **textViews** en blanc. Puis vous allez ajouter des gestionnaires d'événements aux **textViews** afin de modifier leur couleur d'arrière-plan. Et la couleur d'arrière-plan du **constraintLayout**.

Ouvrez le fichier **res/values/themes.xml** et modifier la valeur du style **whiteBox** en blanc. Les **textViews** auront la couleur blanche au départ, puis changeront de couleur quand l'utilisateur va les toucher.

```
<item name="android:background">@android:color/white</item>
```

Ouvrez le fichier **MainActivity.kt**, ajoutez la fonction **makeColored()** après la fonction **onCreate()**.

```
private fun makeColored(view: View) {  
}
```

Chacun des **textViews** créé dans le fichier **activity_main.xml** possède un **id**. Pour définir une couleur, le code basculera à l'aide de l'instruction **when** sur l'**ID** de ressource de la vue. C'est un modèle courant d'utiliser une fonction de gestionnaire de clic pour de nombreuses vues lorsque l'action de clic est la même.

Modifiez la fonction `makeColored()` comme suit en utilisant la fonction `setBackgroundColor` sur chaque `textView`.

```
private fun makeColored(view: View) {
    when (view.id) {

        // textViews using Color class colors for the background
        R.id.tvCGodin -> view.setBackgroundColor(Color.DKGRAY)
        R.id.tvName -> view.setBackgroundColor(Color.GRAY)
        R.id.tvCours1 -> view.setBackgroundColor(Color.BLUE)
        R.id.tvCours2 -> view.setBackgroundColor(Color.MAGENTA)
        R.id.tvCours3 -> view.setBackgroundColor(Color.BLUE)
        else -> view.setBackgroundColor(Color.LTGRAY)
    }
}
```

Ouvrez le fichier `activity_main.xml`, donnez un id `constraint_layout` à la vue racine `constraintLayout`.

Ouvrez le fichier `MainActivity.kt` et ajoutez la fonction `setListeners()` suivante après la fonction `makeColored()`. Utilisez `findViewById` pour obtenir une référence pour chacune des vues définies dans le fichier `activity_main.xml`.

```
private fun setListeners() {

    val tvCGodin = findViewById<TextView>(R.id.tvCGodin)
    val tvName = findViewById<TextView>(R.id.tvName)
    val tvCours1 = findViewById<TextView>(R.id.tvCours1)
    val tvCours2 = findViewById<TextView>(R.id.tvCours2)
    val tvCours3 = findViewById<TextView>(R.id.tvCours3)

    val rootConstraintLayout = findViewById<View>(R.id.constraint_layout)
}
```

Ajoutez les lignes suivantes à la fin de la fonction `setListeners`. Vous vous créez une liste des vues définies précédemment.

```
val clickableViews: List<View> =
    listOf(
        tvCGodin, tvName, tvCours1,
        tvCours2, tvCours3, rootConstraintLayout
    )
```

Ajoutez un écouteur pour chacune des vues ajoutées à la liste comme suit:

```
for (item in clickableViews) {  
    item.setOnClickListener { makeColored(it) }  
}
```

Ajoutez la ligne suivante à la fin de la fonction onCreate().

```
setListeners()
```

Exécutez votre application. Au début, vous allez avoir un écran vide. Cliquez ou touchez votre écran pour révéler les cases et l'arrière-plan.

Exercice

Utilisez des images à la place du texte avec une couleur d'arrière-plan. L'application doit révéler des images quand l'utilisateur appuie sur les **textViews**.

Astuce: Ajoutez les images dans le répertoire **drawable** pour en faire des ressources pouvant être dessinées. Utilisez la fonction **setBackgroundResource()** pour définir une image comme arrière-plan d'un **textView**.

Exemple:

```
R.id.tvCours2 ->  
view.setBackgroundResource(R.drawable.ic_launcher_background)
```


Partie 2

Créez un nouveau projet **Laboratoire1b** dans Android Studio.

1. Copier le morceau de code suivant dans le fichier **activity_main.xml** (onglet **code**) **avant** la ligne `</ConstraintLayout>`

```
<TextView
    android:id="@+id/textView_img"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    android:layout_marginLeft="8dp"
    android:layout_marginRight="8dp"
    android:layout_marginTop="8dp"
    android:text="@string/message_textView_img"
    android:textAlignment="center"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintHorizontal_bias="0.0"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    tools:text="@string/message_textView_img" />
```

A consulter pour mieux comprendre:

<https://developer.android.com/reference/android/support/constraint/ConstraintLayout.html>

2. Ajoutez la ligne suivante au fichier **strings.xml** :

```
<string name="message_textView_img"> Ceci est une zone de texte
avec 4 images</string>
```

Observez le résultat en mode *Design*. Vous pouvez exécuter à nouveau pour voir le résultat sur votre téléphone/tablette. Le nouveau composant apparaît complètement en bas dans l'interface. Utiliser la fenêtre de propriétés du nouveau composant pour comprendre les différents attributs.

3. Relancez votre application.
4. Elle doit ressembler à ceci :



5. Complétez le code XML ci-dessus pour mettre le texte du nouveau **textView** en **gras**, changer sa couleur à **blanc**, son arrière-plan à **holo_blue_dark** (déjà présent dans Android Studio) et sa taille à **16sp** (à insérer dans le fichier **dimens.xml**). Utilisez un style comme dans le projet 1 pour ce faire. Le texte doit être centré.

2. Ajout d'un champ de texte de saisie par programmation

Donner un id rootLayout au ConstraintLayout se trouvant à la racine. Donnez un id aussi à votre premier textView contenant le texte "Hello World"

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/rootLayout"
```

6. Sauvegarder le fichier **activity_main.xml**
7. Modifiez la fonction **onCreate** comme suit dans la classe **MainActivity.kt**.

```

public override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val constraintLayout = findViewById<ConstraintLayout>(R.id.rootLayout)
    val editText = EditText(this)
    editText.id = View.generateViewId()

    editText.hint="Tapez votre texte ici"
    editText.layoutParams = ConstraintLayout.LayoutParams(0, ConstraintLayout.LayoutParams.WRAP_CONTENT)

    constraintLayout.addView(editText)
    val constraintSet = ConstraintSet()
    constraintSet.clone(constraintLayout)
    constraintSet.connect(editText.id,ConstraintSet.TOP, R.id.textView_img, ConstraintSet.BOTTOM,20)
    constraintSet.connect(editText.id,ConstraintSet.LEFT, ConstraintSet.PARENT_ID, ConstraintSet.LEFT,20)
    constraintSet.applyTo(constraintLayout)
}

```

Noter l'utilisation de l'instruction :

```
val constraintLayout = findViewById<ConstraintLayout>(R.id.rootLayout)
```

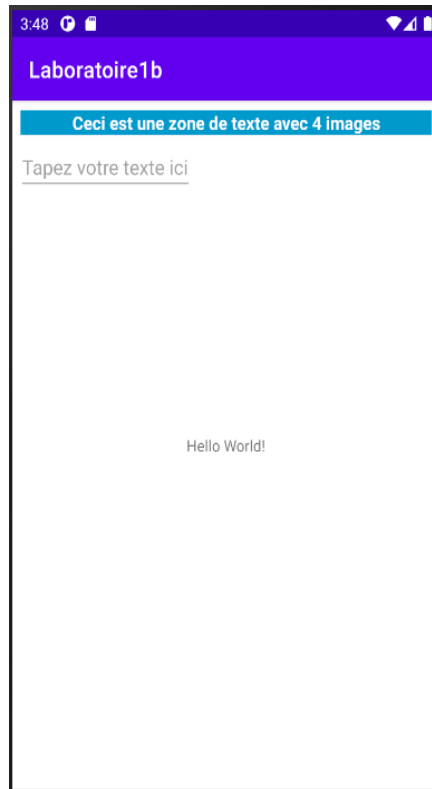
Elle permet de récupérer l'identifiant d'un composant de votre fichier xml afin de l'utiliser dans **Kotlin**.

Vous devez aussi ajouter la ligne suivante avant la ligne `</resources>` à votre fichier strings.xml pour pouvoir donner un identifiant à votre `editText`.

```
<item name="myEditText" type="id" />
```

8. Exécuter.

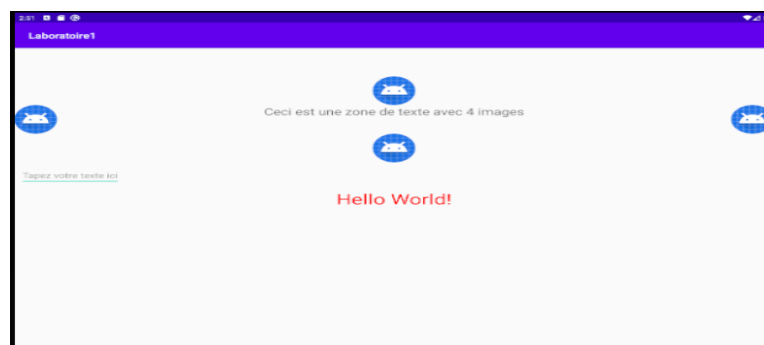
Votre application doit ressembler à ceci : (le clavier apparaît de manière systématique pour la saisie)



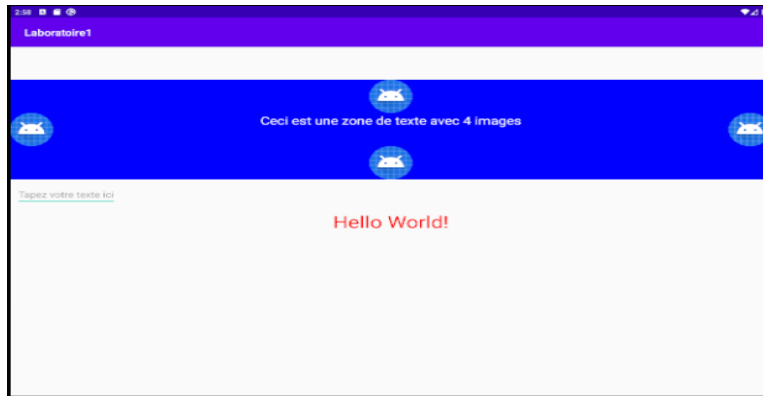
Modifiez l'application pour aboutir à l'interface ci-dessous.

Utilisez les attributs du ***drawableStart***, ***drawableEnd***, ***drawableTop*** et ***drawableBottom*** pour ajouter les images à votre ***TextView***.

Vous devez avoir le résultat suivant quand vous exécutez à nouveau.



Modifiez par la suite l'arrière-plan du textView pour avoir un fond bleu et le texte en blanc comme suit:



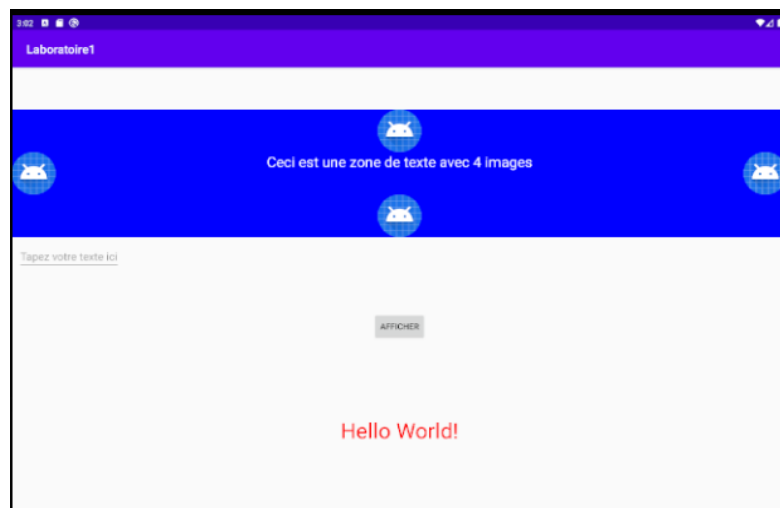
Modifiez l'application pour que le champ **EditText** soit centré dans l'interface.

3. Ajout d'un bouton en utilisant l'interface graphique (drag and drop)

Conseil : créez des contraintes vers le parent et, si besoin, vers les vues voisines pour stabiliser l'emplacement (paysage/portrait).

9. Cliquer sur activity_main.xml. Choisir **Design**.
10. À partir du panneau **Palette**, développer **Widgets**, sélectionner le composant **Button** et glissez-le sur le *layout*. Ajuster les propriétés du bouton pour avoir ce qui suit à l'exécution:

Le bouton Afficher doit être placé au milieu dans le sens horizontal et au milieu des deux TextView dans le sens vertical



4. Interactions avec les composants

Nous allons programmer le clic sur le bouton **Afficher**, pour afficher le message saisi dans la zone de texte **editText** créée par programmation. Dans les deux cas qui suivent, il faut

d'abord récupérer l'identifiant du bouton, puis programmer l'écouteur d'événement qui réalisera l'affichage

4.1 Utilisation d'un objet Toast

Un Toast est un message qui s'affiche pendant quelques secondes en premier plan dans une fenêtre dont la taille est imposée par la taille du message. Intéressant dans le cas des messages purement informatifs.

- i. Copier le code suivant dans MainActivity.java

```
val btnAfficher = findViewById<Button>(R.id.button)
btnAfficher.setOnClickListener {
    val context = applicationContext
    val text = editText.text
    val duree = Toast.LENGTH_LONG
    val toast = Toast.makeText(context, text, duree)
    toast.show()
}
```

- ii. Importez les classes nécessaires
- iii. Exécuter, entrer du texte dans la zone de saisie, puis cliquer sur le bouton Afficher. Observer le résultat.

Note: Dans les cas d'un bouton créé à l'aide de l'interface graphique, il est aussi possible d'associer, dans la fenêtre de propriétés, un nom de méthode à l'événement onClick, puis de programmer son corps dans l'activité correspondante. L'entête de la méthode serait :

public void nom_methode(View v).

4.2 Utilisation d'une boîte de dialogue

Il s'agit d'utiliser une boîte de dialogue d'alerte très simplifiée.

- iv. Mettre en commentaires l'écouteur d'événement du bouton **Afficher** créé précédemment.
- v. Copier et bien comprendre le code suivant dans *MainActivity*

```
btnAfficher.setOnClickListener{
    val alertDialog = AlertDialog.Builder(this)
    alertDialog.setTitle(" Texte saisi ");
    alertDialog.setMessage(editText.text);
    alertDialog.setIcon(android.R.mipmap.sym_def_app_icon);

    alertDialog.setPositiveButton("OK") { _, _ ->
        val toast = Toast . makeText (this, "vous avez choisi OK", Toast.LENGTH_LONG);
        toast.show();
    }

    alertDialog.setNegativeButton("Annuler") { _, _ ->
        val toast = Toast . makeText (this, "vous avez choisi Annuler", Toast.LENGTH_LONG);
        toast.show();
    }
}
```

```
} alertDialog.show()
```

5. À remettre

Checklist de remise :

- Le projet compile sans erreurs.
- L'interface correspond aux captures (téléphone et tablette).
- Les interactions demandées fonctionnent (clics, Toast/AlertDialog).
- Les ressources (strings, colors, dimens) sont utilisées correctement.
- Le fichier .zip porte votre nom et le code du cours.
 - L'application compressée dans un fichier zippé.
 - Au plus tard, **Mercredi, le 1^{er} septembre 2025 à 9:00.**
 - AUCUN RETARD ACCEPTÉ. LES EXERCICES REMIS EN RETARD NE SERONT PAS CORRIGÉS.
 - Sur LÉA